

Trabalhando com Arquivos em Java

Parte 1 — Fundamentos: File, Path, Files e Exceções

Apostila completa sobre o modelo de arquivos do Java, do legado java.io ao NIO.2, incluindo o tratamento correto de exceções ao lidar com recursos externos. Exemplos práticos aplicados a um sistema de arquivos de uma biblioteca digital.

CONTEÚDO DA APOSTILA

- 01** Comunicando-se com Recursos Externos
- 02** File, Path e Files: do Modelo Legado ao NIO.2
- 03** Funcionalidades em Comum entre File e Files
- 04** Atributos e Métodos do Path
- 05** Percorrendo o Sistema de Arquivos
- 06** Renomeando, Copiando e Deletando
- 07** Exceções: Checked, Unchecked e o finally
- 08** Try-with-Resources

Sumário

0 1 Comunicando-se com Recursos Externos

O que acontece de fato quando o Java abre um arquivo, e por que fechar recursos importa

0 2 File, Path e Files: do Modelo Legado ao NIO.2

A diferença entre o antigo File e a dupla Path/Files introduzida no NIO.2

0 3 Funcionalidades em Comum entre File e Files

Tabela comparativa de operações e as vantagens de desempenho do NIO.2

0 4 Atributos e Métodos do Path

Como consultar metadados de um arquivo: tamanho, datas e tipo

0 5 Percorrendo o Sistema de Arquivos

list, walk, find e walkFileTree para navegar em árvores de diretórios

0 6 Renomeando, Copiando e Deletando

Operações de manutenção sobre arquivos e diretórios com Files

0 7 Exceções: Checked, Unchecked e o finally

IOException, LBYL vs EAFP, e o papel real da cláusula finally

0 8 Try-with-Resources

Fechamento automático de recursos e a hierarquia de Throwable

Comunicando-se com Recursos Externos

O que realmente acontece quando uma aplicação Java abre um arquivo

Toda vez que uma aplicação Java precisa trocar informações com o mundo fora da JVM — um arquivo em disco, uma conexão de rede, um banco de dados ou um socket — ela está lidando com um **recurso**. Diferente de um objeto comum, criado e descartado inteiramente dentro da memória da própria JVM, um recurso externo envolve o sistema operacional, e isso muda as regras do jogo: pode falhar por motivos fora do controle do programa, e precisa ser devidamente liberado depois de usado.

É exatamente por lidar com esse tipo de imprevisibilidade que o tratamento de exceções se torna muito mais relevante quando o assunto é I/O (Input/Output). Um arquivo pode não existir, o processo pode não ter permissão de leitura, o disco pode estar cheio — nenhuma dessas situações pode ser prevista apenas olhando o código-fonte.

O que o Java faz ao abrir um arquivo

Quando uma classe de acesso a arquivos é instanciada, o Java delega a operação para o sistema operacional, que executa uma sequência de verificações: primeiro confirma se o arquivo existe; em caso positivo, verifica se o usuário ou processo tem a permissão necessária para o tipo de acesso solicitado; por fim, recupera os metadados do arquivo e aloca um *file descriptor* — um handle que representa o arquivo aberto.

Na sequência, o sistema operacional registra uma entrada na sua tabela de arquivos abertos, pode aplicar um bloqueio (lock) sobre o arquivo, e frequentemente reserva memória para um buffer, cacheando parte ou todo o conteúdo do arquivo para otimizar as próximas leituras e escritas.

Fechando um recurso de arquivo

Boa parte das classes Java para arquivos faz a abertura parecer a simples instancição de qualquer outro objeto — e é justamente por isso que é fácil esquecer que, por trás, um recurso real foi aberto no sistema operacional. Fechar esse handle libera a memória usada para armazenar dados relacionados ao arquivo e permite que outros processos voltem a acessá-lo.

DICA

Felizmente, a maioria das classes Java usadas para interagir com arquivos implementa a interface **AutoCloseable**, o que torna o fechamento de recursos praticamente automático quando combinado com o try-with-resources, que veremos no capítulo 08.

IO, NIO e NIO.2: desfazendo a confusão de nomes

O Java acumulou, ao longo de várias versões, um conjunto grande de classes e pacotes para lidar com entrada e saída de dados. Essa evolução histórica explica a existência de três termos parecidos, mas com significados distintos.

Termo	Significado	Quando surgiu
IO	java.io — o conjunto original de tipos para leitura e escrita em recursos externos	Java 1.0
NIO	Non-blocking IO — comunicação via Channels e dados em Buffers	Java 1.4
NIO.2	New IO — grandes melhorias, sobretudo no pacote java.nio.file (Path, Files)	Java 1.7

Nos próximos capítulos, o foco estará no legado **java.io** e no moderno **java.nio.file** (NIO.2) — as duas formas mais comuns de trabalhar com arquivos no dia a dia de uma aplicação Java.

File, Path e Files: do Modelo Legado ao NIO.2

Duas gerações de APIs para representar e manipular arquivos

O modelo legado: a classe File

Desde a primeira versão do Java, a classe **File** (e sua irmã **FileReader**) faz parte da linguagem. A **FileReader** implementa a interface **AutoCloseable** através de sua superclasse **Reader**, e abre um recurso de arquivo de forma implícita assim que é instanciada. A classe **File** se comporta de modo diferente: criar uma instância de **File** **não abre** o arquivo — você está, na prática, trabalhando com um *file handle*, uma referência que permite executar operações no estilo do sistema operacional (verificar existência, tamanho, permissões etc.), sem necessariamente acessar o conteúdo do arquivo.

File handle x recurso de arquivo

Um **file handle** é uma representação abstrata do arquivo, usada pelo sistema operacional para rastreá-lo — ele não contém nenhum dado real do arquivo. Já o **recurso de arquivo** é o conteúdo de fato, armazenado em disco e acessível tanto pelo sistema operacional quanto pelas aplicações.

Conceitos de sistema de arquivos

Conceito	Descrição
Diretório	Um contêiner do sistema de arquivos para outros diretórios ou arquivos
Path	Um diretório ou nome de arquivo, podendo incluir informações sobre os diretórios pai
Diretório raiz	O diretório de nível mais alto em um sistema de arquivos
Diretório de trabalho atual	O diretório a partir do qual o processo em execução está rodando
Path absoluto	Inclui a raiz — começa com / (ou C:\ no Windows)
Path relativo	Definido em relação ao diretório de trabalho atual — não começa com /

O modelo atualizado: Path, Paths e Files (NIO.2)

A partir do JDK 7, o NIO.2 trouxe uma nova forma de representar e manipular arquivos. **Path** é uma interface — e não uma classe, como **File**. A classe **Paths** reúne exclusivamente métodos estáticos que retornam uma instância de **Path**, enquanto a classe **Files** concentra dezenas de métodos estáticos que realizam operações sobre arquivos e diretórios.

Duas formas de trabalhar: File x Files/Path

Com a classe **File** (estilo antigo), você obtém uma instância a partir de um construtor e executa métodos diretamente nela — o comportamento é membro da própria instância. Já no NIO.2, o fluxo é diferente: primeiro

obtemos uma instância de Path (usando métodos de fábrica da própria interface Path, da classe Paths, ou de outros tipos), e depois passamos esse Path como argumento para um método estático da classe Files, que executa a operação desejada.

```
// Catálogo da biblioteca digital – estilo antigo (java.io)
File catalogo = new File("biblioteca/catalogo.txt");
if (catalogo.exists()) {
    System.out.println("Tamanho: " + catalogo.length() + " bytes");
}
// Catálogo da biblioteca digital – estilo NIO.2
Path caminho = Paths.get("biblioteca", "catalogo.txt");
if (Files.exists(caminho)) {
    long tamanho = Files.size(caminho);
    System.out.println("Tamanho: " + tamanho + " bytes");
}
```

NOTA

Repare no padrão: no modelo antigo, o comportamento pertence ao objeto (*catalogo.length()*). No NIO.2, o comportamento pertence à classe Files, que recebe o Path como argumento (*Files.size(caminho)*).

Funcionalidades em Comum entre File e Files

Comparando operações equivalentes nas duas gerações da API

Praticamente toda operação disponível na antiga classe `File` tem um equivalente estático na classe `Files`, recebendo um `Path` como argumento. A tabela abaixo resume o mapeamento mais usado no dia a dia.

Funcionalidade	Método de instância (<code>File</code>)	Método estático (<code>Files</code> , com <code>Path</code>)
Criar arquivo	<code>createNewFile()</code>	<code>createFile(Path p)</code>
Deletar arquivo/diretório	<code>delete()</code>	<code>delete(Path p) / deleteIfExists(Path p)</code>
Verificar tipo de path	<code>isDirectory()</code> / <code>isFile()</code>	<code>isDirectory(Path p) / isRegularFile(Path p)</code>
Tamanho em bytes	<code>length()</code>	<code>size(Path p)</code>
Listar diretório	<code>listFiles()</code>	<code>list(Path p)</code>
Criar diretório(s)	<code>mkdir()</code> / <code>mkdirs()</code>	<code>createDirectory(Path p) / createDirectories(Path p)</code>
Renomear	<code>renameTo(File dest)</code>	<code>move(Path src, Path dest)</code>

Por que preferir o NIO.2 hoje

As operações do NIO.2 foram significativamente ampliadas em relação ao modelo legado. Entre as melhorias mais relevantes estão:

Recurso do NIO.2	O que oferece
I/O assíncrono	Operações de arquivo que não bloqueiam a thread chamadora
Locking granular	Permite travar apenas uma região do arquivo, não o arquivo inteiro
Metadados	Recuperação rica de atributos do arquivo (datas, tamanho, tipo)
Links simbólicos	Suporte nativo à criação e manipulação de symlinks
Notificações	Um path pode ser observado, avisando serviços registrados sobre mudanças

Além de mais recursos, o NIO.2 também tende a ter **melhor desempenho**: seus tipos são non-blocking, o que permite acesso assíncrono por múltiplas threads; a gestão de memória é mais eficiente, lendo e escrevendo diretamente entre arquivo e memória por meio de um **FileChannel**; e é possível ler ou escrever em múltiplos buffers numa única operação.

Atributos e Métodos do Path

Consultando metadados de arquivos e diretórios com Files

Além de criar, mover e apagar arquivos, é comum precisar consultar informações sobre eles: quando foram modificados pela última vez, se são diretórios ou arquivos regulares, ou qual o tamanho ocupado em disco. O NIO.2 concentra essa leitura de metadados nos métodos **getAttribute** e **getAttributes** da classe Files.

Literal (String) para Files.getAttribute	Tipo retornado	Método alternativo
"lastModifiedTime"	FileTime	Files.getLastModifiedTime(...)
"lastAccessTime"	FileTime	—
"creationTime"	FileTime	—
"size"	Long	Files.size(...)
"isRegularFile"	Boolean	Files.isRegularFile(...)
"isDirectory"	Boolean	Files.isDirectory(...)
"isSymbolicLink"	Boolean	Files.isSymbolicLink(...)
"isOther"	Boolean	—

Como o método **Files.getAttribute** devolve os dados na forma de **Object**, geralmente é necessário fazer um cast explícito para o tipo esperado. Para os atributos mais usados, porém, a classe Files já oferece métodos específicos, dispensando esse cast — como mostrado na última coluna da tabela acima.

```
// Consultando metadados de um item do acervo da biblioteca digital
Path arquivo = Paths.get("acervo", "1234-resumo.pdf");
// Via método específico, sem necessidade de cast:
FileTime modificadoEm = Files.getLastModifiedTime(arquivo);
long tamanhoBytes = Files.size(arquivo);
// Via getAttribute, retorna Object — exige cast:
Object valor = Files.getAttribute(arquivo, "isDirectory");
boolean ehDiretorio = (Boolean) valor;
// Lendo várias views de atributos de uma vez:
Map<String, Object> atributos = Files.readAttributes(arquivo, "*");
System.out.println(atributos.get("size"));
```


DICA

Prefira sempre os métodos nomeados (`Files.size`, `Files.isDirectory`, `Files.getLastModifiedTime`) quando eles existirem — o código fica mais legível e você evita casts desnecessários.

Percorrendo o Sistema de Arquivos

list, walk, find e walkFileTree para navegar em árvores de diretórios

Padrões glob para filtrar arquivos

Um **glob** é uma linguagem de padrões simplificada, parecida com expressões regulares, mas com uma sintaxe muito mais enxuta — ideal para filtrar arquivos por extensão ou nome. O próprio `java.nio.file.FileSystem` expõe um `PathMatcher` construído a partir de um padrão glob ou regex.

```
// Filtrando apenas arquivos PDF no acervo, usando um glob
PathMatcher matcher = FileSystems.getDefault()
    .getPathMatcher("glob:*.pdf");
try (DirectoryStream<Path> stream =
    Files.newDirectoryStream(Paths.get("acervo"), "*.pdf")) {
    for (Path pdf : stream) {
        System.out.println(pdf.getFileName());
    }
}
```

Files.list, Files.walk e Files.find

Para percorrer diretórios de forma programática, a classe `Files` oferece três métodos que retornam um `Stream<Path>`: **list** lê apenas o nível mais raso de um diretório; **walk** percorre recursivamente toda a árvore, até uma profundidade opcional; e **find** faz o mesmo que `walk`, mas já aplica um filtro (um `BiPredicate`) durante a travessia, o que costuma ser mais eficiente quando você já sabe o que está procurando.

```
// Somando o tamanho de todos os PDFs do acervo, recursivamente
try (Stream<Path> caminhos = Files.walk(Paths.get("acervo"))) {
    long totalBytes = caminhos
        .filter(Files::isRegularFile)
        .filter(p -> p.toString().endsWith(".pdf"))
        .mapToLong(p -> {
            try {
                return Files.size(p);
            } catch (IOException e) {
                return 0L;
            }
        })
        .sum();
    System.out.println("Total: " + totalBytes + " bytes");
}
```

Files.walkFileTree e o FileVisitor

Quando o percurso precisa de mais controle — por exemplo, executar uma ação diferente ao entrar e ao sair de cada diretório — a classe `Files` oferece o método **`walkFileTree`**. Assim como `walk`, ele percorre a árvore em profundidade (depth-first): o código visita recursivamente todos os elementos filhos antes de passar para os diretórios irmãos. A alternativa seria uma travessia em largura (breadth-first), na qual os nós dependentes só são visitados depois dos nós irmãos.

Por ser depth-first, `walkFileTree` oferece um mecanismo natural para acumular informações dos filhos até o pai — por exemplo, somar o tamanho de uma pasta inteira. Para isso, o Java expõe pontos de entrada na travessia através da interface **`FileVisitor`**, que define quatro eventos: antes de visitar um diretório, depois de visitar um diretório, ao visitar um arquivo, e quando há falha ao visitar um arquivo.

A interface **`SimpleFileVisitor`** é a implementação padrão mais simples de `FileVisitor`, com todos os métodos já implementados de forma neutra — basta sobrescrever apenas os que interessam. Em todos os métodos, o retorno é um valor do enum **`FileVisitResult`** (`CONTINUE`, `SKIP_SUBTREE`, `SKIP_SIBLINGS` ou `TERMINATE`), que controla o rumo da travessia.

```
// Calculando o tamanho total e a contagem de arquivos do acervo
Map<String, Long> resumo = new HashMap<>();
resumo.put("tamanhoTotal", 0L);
resumo.put("totalArquivos", 0L);
Files.walkFileTree(Paths.get("acervo"), new SimpleFileVisitor<Path>() {
    @Override
    public FileVisitResult visitFile(Path arquivo, BasicFileAttributes attrs) {
        resumo.merge("tamanhoTotal", attrs.size(), Long::sum);
        resumo.merge("totalArquivos", 1L, Long::sum);
        return FileVisitResult.CONTINUE;
    }
});
```

Renomeando, Copiando e Deletando

Operações de manutenção sobre arquivos e diretórios com Files

Grande parte do trabalho do dia a dia com arquivos não é ler nem escrever seu conteúdo, mas sim organizá-los: renomear, copiar, mover e apagar arquivos e diretórios inteiros. Ocasionalmente também surge a necessidade de fazer uma busca e substituição global no conteúdo de um arquivo já existente.

```
// Manutenção do acervo da biblioteca digital com NIO.2
Path origem = Paths.get("acervo", "rascunho.txt");
Path destino = Paths.get("acervo", "publicado.txt");
// Renomear/mover um arquivo:
Files.move(origem, destino, StandardCopyOption.REPLACE_EXISTING);
// Copiar um arquivo:
Path backup = Paths.get("backup", "publicado.txt");
Files.copy(destino, backup, StandardCopyOption.REPLACE_EXISTING);
// Deletar, exigindo que o arquivo exista:
Files.delete(Paths.get("acervo", "obsoleto.txt"));
// Deletar sem lançar exceção se o arquivo já não existir:
Files.deleteIfExists(Paths.get("acervo", "temp.txt"));
// Criar diretórios aninhados de uma vez:
Files.createDirectories(Paths.get("acervo", "2026", "julho"));
```

ATENÇÃO

`Files.delete` lança uma exceção se o arquivo ou diretório não existir, ou se um diretório não estiver vazio. Já `Files.deleteIfExists` simplesmente não faz nada quando o alvo já não existe — escolha o método de acordo com o comportamento que sua aplicação espera.

Busca e substituição no conteúdo de um arquivo

Como o conteúdo de um arquivo de texto pode ser lido inteiro como uma única `String`, uma substituição global pode ser feita combinando a leitura completa, o método **`replace`** (ou uma expressão regular) e uma nova escrita.

```
// Substituindo um termo em todos os registros do catálogo
Path catalogo = Paths.get("acervo", "catalogo.txt");
String conteudo = Files.readString(catalogo);
String atualizado = conteudo.replace("Editora Antiga", "Editora Nova");
Files.writeString(catalogo, atualizado);
```

Exceções: Checked, Unchecked e o finally

Entendendo IOException, LBYL vs EAFP, e o papel real da cláusula finally

IOException: uma exceção checked

A **IOException** é uma *checked exception* — a superclasse de muitas exceções comuns encontradas ao trabalhar com recursos externos. Uma checked exception representa um problema previsível ou comum. Um erro de digitação no nome do arquivo, por exemplo, é comum o suficiente para o Java ter uma exceção nomeada especificamente para essa situação: a **FileNotFoundException**, subclasse de **IOException**.

Como tratar uma checked exception

Uma checked exception precisa ser tratada ou declarada — o código simplesmente não compila se isso for ignorado. Existem duas alternativas: envolver a instrução que lança a exceção num bloco try/catch e tratar a situação ali mesmo, ou alterar a assinatura do método, declarando uma cláusula throws com esse tipo de exceção, repassando a responsabilidade para quem chamar o método.

```
// Duas formas de lidar com uma checked exception
// 1) Tratando localmente:
public void carregarCatalogo(String caminho) {
    try {
        String conteudo = Files.readString(Paths.get(caminho));
        System.out.println(conteudo);
    } catch (IOException e) {
        System.out.println("Não foi possível ler o catálogo: " + e.getMessage());
    }
}

// 2) Repassando a responsabilidade para o chamador:
public String carregarCatalogo(String caminho) throws IOException {
    return Files.readString(Paths.get(caminho));
}
```

LBYL ou EAFP?

Existem duas filosofias comuns para lidar com operações que podem falhar. **LBYL** (Look Before You Leap — "olhe antes de saltar") verifica condições de erro antes de executar a operação. **EAFP** (Easier to Ask Forgiveness than Permission — "mais fácil pedir perdão do que permissão") assume que a operação normalmente terá sucesso, e trata os erros apenas se eles de fato ocorrerem.

Característica	LBYL	EAFP
Abordagem	Verifica erros antes de executar a operação	Assume sucesso e trata erros se ocorrerem
Vantagens	Pode ser mais eficiente se erros forem raros	Pode ser mais conciso e legível
Desvantagens	Pode ser mais verboso se erros forem comuns	Pode ser mais difícil de depurar se o erro for inesperado

Como em quase toda pergunta sobre design de software, a resposta correta é: depende. Em um sistema de arquivos, onde a existência e as permissões de um arquivo podem mudar entre a verificação e o uso, o estilo EAFP costuma ser mais seguro — por isso o próprio Java tende a favorecer o try/catch nesse contexto.

Reconhecendo uma checked exception

Numa IDE, é fácil identificar uma checked exception: o código simplesmente não compila se ela não for tratada ou declarada. Por definição, uma checked exception é qualquer exceção que **não** seja unchecked. Uma **unchecked exception**, por sua vez, é toda instância de **RuntimeException** ou de uma de suas subclasses — e o compilador não exige nenhum tratamento explícito para elas.

A cláusula finally

A cláusula **finally** é usada junto de um bloco try. Um try tradicional exige pelo menos um catch ou um finally (ou os dois). Quando presente, o finally é sempre declarado depois do catch, e seu código é executado **sempre**, independentemente do que aconteça no try ou no catch — inclusive se uma exceção não tratada for lançada. Vale notar que o bloco finally não tem acesso às variáveis locais declaradas dentro do try ou do catch.

Historicamente, o propósito do finally era concentrar num único bloco as operações de limpeza: fechar conexões abertas, liberar locks e/ou liberar recursos — garantindo que esse código rodasse tanto em uma finalização normal quanto em caso de exceção. Hoje, para o fechamento de recursos, o try-with-resources (introduzido no JDK 7) é a abordagem recomendada, deixando o finally mais livre para outras tarefas relevantes, como logging ou atualização de interface.

ATENÇÃO

Usar o finally em excesso tem desvantagens: pode tornar o código mais difícil de ler, pode esconder erros — dificultando a depuração — e, se usado para tarefas não relacionadas à limpeza de recursos, prejudica a manutenção do código.

Try-with-Resources

Fechamento automático de recursos e a hierarquia de Throwable

O try-with-resources, introduzido no JDK 7, recebe uma lista de variáveis de recurso separadas por ponto e vírgula, declaradas entre parênteses logo após a palavra-chave try. Cada recurso dessa lista precisa implementar a interface **AutoCloseable** (ou sua subinterface Closeable).

```
// Comparando try-with-resources com o try tradicional
// try-with-resources – fechamento automático:
try (BufferedReader leitor =
Files.newBufferedReader(Paths.get("acervo", "resenha.txt"))) {
String linha;
while ((linha = leitor.readLine()) != null) {
System.out.println(linha);
}
} catch (IOException e) {
System.out.println("Erro ao ler resenha: " + e.getMessage());
}
// try tradicional – fechamento manual, com finally:
BufferedReader leitor = null;
try {
leitor = Files.newBufferedReader(Paths.get("acervo", "resenha.txt"));
String linha;
while ((linha = leitor.readLine()) != null) {
System.out.println(linha);
}
} catch (IOException e) {
System.out.println("Erro ao ler resenha: " + e.getMessage());
} finally {
if (leitor != null) {
try {
leitor.close();
} catch (IOException e) {
// erro ao fechar, geralmente ignorado ou logado
}
}
}
```

Repare como o try-with-resources elimina completamente a necessidade do finally apenas para fechar o recurso: todos os recursos declarados na lista são fechados automaticamente quando o bloco try termina — seja normalmente, seja por causa de uma exceção.

A hierarquia de Throwable

No topo da hierarquia de exceções do Java está **Throwable**, que se divide em duas ramificações principais: **Error**, que sinaliza problemas sérios que uma aplicação bem-comportada normalmente não deveria tentar

capturar ou recuperar (como um `OutOfMemoryError`); e **Exception**, dividida entre as exceções checked (tudo que não estende `RuntimeException`) e as unchecked (`RuntimeException` e suas subclasses). Essa distinção entre checked e unchecked se torna especialmente relevante quando se usa a segunda variação da cláusula `catch`, com múltiplos tipos de exceção separados por pipe (`|`).

```
// Multi-catch: tratando mais de um tipo de exceção no mesmo bloco
try {
    Path caminho = Paths.get("acervo", "indice.dat");
    long posicao = calcularPosicaoRegistro(caminho, 42);
} catch (IOException | NumberFormatException e) {
    System.out.println("Falha ao localizar registro: " + e.getMessage());
}
```

Fim de apostila

Este capítulo consolidou o modelo de arquivos do Java: a diferença entre `File` e a dupla `Path/Files` do NIO.2, como navegar e manipular o sistema de arquivos, e como tratar exceções de forma robusta ao lidar com recursos externos. Esses fundamentos preparam o terreno para a próxima apostila, que aborda a leitura e a escrita de dados em arquivos.

JavaStudiesHub